



**KESHAV MEMORIAL INSTITUTE OF TECHNOLOGY
(AN AUTONOMOUS INSTITUTION)**

Accredited by NBA & NAAC, Approved by AICTE, Affiliated to JNTUH,
Narayanguda, Hyderabad – 500029



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

LAB RECORD

OPERATING SYSTEMS LAB

B.Tech. II YEAR II SEM (KR24)

ACADEMIC YEAR 2025 – 26



**KESHAV MEMORIAL INSTITUTE OF TECHNOLOGY
(AN AUTONOMOUS INSTITUTION)**



**Accredited by NBA & NAAC, Approved by AICTE, Affiliated to JNTUH, Hyderabad
Department of Computer Science & Engineering**

Certificate

This is to certify that following is a Bonafide Record of the workbook task done by

_____ bearing Roll No _____ of _____ Branch of
_____ year B.Tech Course in the _____ Subject during the
Academic year _____ & _____ under our supervision.

Number of week tasks completed: _____

Signature of Staff Member Incharge

Signature of Head of the Dept.

Signature of Internal Examiner

Signature of External Examiner



INDEX

Exp No:	List of Experiments													
1	a) Identify the peripherals of a computer, components in a CPU and its functions. Draw the block diagram of the CPU along with the configuration of each peripheral. b) Disassemble and assemble the PC back to working condition. c) Install MS windows on the personal computer. d) Install Linux on the computer. This computer should have windows installed. The system should be configured as dual boot with both windows and Linux													
2	Installation of Docker and Ubuntu OS.													
3	a) Implement in C language the following Unix commands using system calls i) cat ii) ls iii) mv b) Write a C program to create child process and allow parent process to display “parent” and the child to display “child” on the screen.													
4	Assume you have the following jobs to execute with one processor, with the jobs arriving in the order listed here: <table><tr><td>i</td><td>T(pi)</td></tr><tr><td>0</td><td>80</td></tr><tr><td>1</td><td>20</td></tr><tr><td>2</td><td>10</td></tr><tr><td>3</td><td>20</td></tr><tr><td>4</td><td>50</td></tr></table> With the following values write a program to get the required output which is listed below Suppose a system uses FCFS scheduling. Create a Gantt chart illustrating the execution of these processes? What is the turnaround time for process p3? What is the average wait time for the processes?	i	T(pi)	0	80	1	20	2	10	3	20	4	50	
i	T(pi)													
0	80													
1	20													
2	10													
3	20													
4	50													
5	a) Write a C program that illustrates communication between two unrelated process using named pipes. b) Write a C program that receives a message from message queue and display them. c) Write a C program to allow cooperating process to lock a resource for exclusive use (using semaphore). d) Write a C program that illustrates the suspending and resuming process using signal. e) Write a C program that implements Producer-Consumer system with two process using semaphore.													
6	Consider the following snapshot of a system. P0, P1, P2, P3, P4 are the processes and A, B, C, D are the resource types. The values in the table indicates the number of instances of a specific resource (for example: 3 3 2 1 under the last column indicates that there are 3 A type, 3 B-type, 2 C-type and 1 D-type resources are available after allocating the resources to all five processes). The numbers under allocation-column indicate that those numbers													

of resources are allocated to various processes mentioned in the first column. The numbers under Max-column indicate the maximum number of resources required by the processes. For example: in 1st row under allocation-column 2 0 0 1 indicate there are 2 A-type, 0 B type, 0 C-type and 1 D-type resources are allocated to process P0. Whereas 4 2 1 2 under Max-column indicate that process P0's maximum requirement is 4 A-type, 2 B-type, 1 C-type and 2 D-type resources.

Process	Allocation	Max	Available
	A B C D	A B C D	A B C D
P0	2 0 0 1	4 2 1 2	3 3 2 1
P1	3 1 2 1	5 2 5 2	
P2	2 1 0 3	2 3 1 6	
P3	1 3 1 2	1 4 2 4	
P4	1 4 3 2	3 6 6 5	

Answer the following questions using banker's algorithm by providing all intermediate steps

How many instances of resources are present in the system under each type of a resource?

Compute the Need matrix for the given snapshot of a system.

Verify whether the snapshot of the present system is in a safe state by demonstrating an order in which the processes may complete. If a request from process P1 arrives for (1,1,0,0), can the request be granted immediately?

If a request from process P4 arrives for (0,0,2,0), can the request be granted immediately?

7

Write a C program to simulate the following memory management technique: Paging.

8

- Write a C program that takes one or more file/directory names as command line input and reports following information: i) File Type ii) Number of Links iii) Time of last Access iv) Read, write and execute permissions.
- Write a C program to list every file in directory, its inode number and file name.



KESHAVMEMORIALINSTITUTE OF TECHNOLOGY

(AN AUTONOMOUS INSTITUTION)



**Accredited by NBA & NAAC, Approved by AICTE, Affiliated to JNTUH,
Narayanguda, Hyderabad-500029**

Department of Computer Science & Engineering

Vision of the Institution:

To be the fountain head of latest technologies, producing highly skilled, globally competent engineers.

Mission of the Institution:

- To provide a learning environment that inculcates problem solving skills, professional, ethical responsibilities, lifelong learning through multi modal platforms and prepare students to become successful professionals.
- To establish Industry Institute Interaction to make students ready for the industry.
- To provide exposure to students on latest hardware and software tools.
- To promote research-based projects/activities in the emerging areas of technology convergence.
- To encourage and enable students to not merely seek jobs from the industry but also to create new enterprises
- To induce a spirit of nationalism which will enable the student to develop, understand India's challenges and to encourage them to develop effective solutions.
- To support the faculty to accelerate their learning curve to deliver excellent service to students



KESHAV MEMORIAL INSTITUTE OF TECHNOLOGY **(AN AUTONOMOUS INSTITUTION)**



**Accredited by NBA & NAAC, Approved by AICTE, Affiliated to JNTUH,
Narayanguda, Hyderabad-500029**

Department of Computer Science & Engineering

Vision of the Department:

To be among the region's premier teaching and research Computer Science and Engineering departments producing globally competent and socially responsible graduates in the most conducive academic environment.

Mission of the Department:

- To provide faculty with state-of-the-art facilities for continuous professional development and research, both in foundational aspects and of relevance to emerging computing trends.
- To impart skills that transform students to develop technical solutions for societal needs and inculcate entrepreneurial talents.
- To inculcate an ability in students to pursue the advancement of knowledge in various specializations of Computer Science and Engineering and make them industry-ready.
- To engage in collaborative research with academia and industry and generate adequate resources for research activities for seamless transfer of knowledge resulting in sponsored projects and consultancy.
- To cultivate responsibility through sharing of knowledge and innovative computing solutions that benefits the society-at-large.
- To collaborate with academia, industry and community to set high standards in academic excellence and in fulfilling societal responsibilities.



KESHAV MEMORIAL INSTITUTE OF TECHNOLOGY (AN AUTONOMOUS INSTITUTION)



Accredited by NBA & NAAC, Approved by AICTE, Affiliated to JNTUH,
Narayanguda, Hyderabad– 500029

Department of Computer Science & Engineering

PROGRAM OUTCOMES (POs)

PO1: Engineering Knowledge: Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.

PO2: Problem Analysis: Identify, formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences and engineering sciences.

PO3: Design / Development of Solutions: Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal and environmental considerations.

PO4: Conduct Investigations of Complex Problems: Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.

PO5: Modern Tool Usage: Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modelling to complex engineering activities with an understanding of the limitations.

PO6: The Engineer and Society: Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.

PO7: Environment and Sustainability: Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development.

PO8: Ethics: Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.

PO9: Individual and Team Work: Function effectively as an individual, and as a member or leader in diverse teams, and in multi disciplinary settings.

PO10: Communication: Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.

PO11: Project Management and Finance: Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.

PO12:Life long Learning: Recognize the need for, and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.



**KESHAV MEMORIAL INSTITUTE OF TECHNOLOGY
(AN AUTONOMOUS INSTITUTION)**

**Accredited by NBA & NAAC, Approved by AICTE, Affiliated to JNTUH,
Narayanguda, Hyderabad-500029**



Department of Computer Science & Engineering

PROGRAM SPECIFIC OUTCOMES (PSOs)

PSO1: An ability to analyze the common business functions to design and develop appropriate Computer Science solutions for social upliftment.

PSO2: Shall have expertise on the evolving technologies like Python, Machine Learning, Deep Learning, Internet of Things (IOT), Data Science, Full stack development, Social Networks, Cyber Security, Big Data, Mobile Apps, CRM, ERP etc.



KESHAV MEMORIAL INSTITUTE OF TECHNOLOGY **(AN AUTONOMOUS INSTITUTION)**

**Accredited by NBA & NAAC, Approved by AICTE, Affiliated to JNTUH,
Narayanguda, Hyderabad-500029**



Department of Computer Science & Engineering

PROGRAM EDUCATIONAL OBJECTIVES (PEOs)

PEO1: Graduates will have successful careers in computer related engineering fields or will be able to successfully pursue advanced higher education degrees.

PEO2: Graduates will try and provide solutions to challenging problems in their profession by applying computer engineering principles.

PEO3: Graduates will engage in life-long learning and professional development by rapidly adapting changing work environment.

PEO4: Graduates will communicate effectively, work collaboratively and exhibit high levels of professionalism and ethical responsibility.



KESHAV MEMORIAL INSTITUTE OF TECHNOLOGY

(AN AUTONOMOUS INSTITUTION)

Accredited by NBA & NAAC, Approved by AICTE, Affiliated to JNTUH, Hyderabad



B.Tech. in COMPUTER SCIENCE AND ENGINEERING

B.Tech. II Year II Semester Syllabus (KR23)

OPERATING SYSTEMS LAB (23CC407PC)

Common to CSE, CSE (DS), CSE (AI & ML), IT

Prerequisites/Corequisites:

1. 23CS108ES – Programming for Problem Solving Lab
2. 23CC403PC – Operating Systems Course

L	T	P	C
0	0	2	1

Course Objectives: The course will help to

1. Understand computer system components, their roles, and the installation of Windows and Linux operating systems in a dual-boot setup.
2. Apply Unix commands and system calls to perform file operations, process creation, and system management tasks.
3. Implement process scheduling algorithms, Inter-Process Communication (IPC) mechanisms, and process synchronization techniques.
4. Analyze deadlock scenarios using the Banker's Algorithm and develop solutions for resource allocation and system stability.
5. Demonstrate memory management techniques, file system operations, and directory management using system calls in a Unix environment.

Course Outcomes: After learning the concepts of this course, the student is able to

1. Illustrate computer hardware components, CPU functions, and OS installation, including dual-boot configuration.
2. Implement Unix commands and system calls for file manipulation, process creation, and inter-process communication.
3. Apply CPU scheduling algorithms, process synchronization methods, and IPC techniques using pipes, message queues, and semaphores.
4. Analyze resource allocation using the Banker's Algorithm and determine system safety through deadlock detection and prevention.
5. Demonstrate memory management techniques like paging and file system operations, including file permissions, directory structures, and system calls for file handling.

List of Exercises

Exercise1:

Identify the peripherals of a computer, components in a CPU and its functions. Draw the block diagram of the CPU along with the configuration of each peripheral.

- a) Disassemble and assemble the PC back to working condition.
- b) Install MS windows on the personal computer.
- c) Install Linux on the computer. This computer should have windows installed. The system should be configured as dual boot with both windows and Linux.

Exercise2:

- Implement in c language the following Unix commands using system calls i)cat ii) ls iii) mv
- Write a C program to create child process and allow parent process to display “parent” and the child to display “child” on the screen

Exercise3:

Assume you have the following job to execute with one processor, with the jobs arriving in the order listed here:

i	T(pi)
0	80
1	20
2	10
3	20
4	50

With the following values write a program to get the required output which is listed below

- Suppose a system uses FCFS scheduling. Create a Gantt chart illustrating the execution of these processes?
- What is the average turnaround time for the processes?
- What is the average wait time for the processes?

Exercise4:

- Write a C program that illustrate communication between two unrelated process using named pipes
- Write a C program that receives a message from message queue and display them
- Write a C program to allow cooperating process to lock a resource for exclusive use (using semaphore)
- Write a C program that illustrate the suspending and resuming process using signal
- Write a C program that implements producer-Consumer system with two process using semaphore

Exercise5:

Consider the following snapshot of a system. P0,P1,P2,P3,P4are the processes and A,B,C,D are the resource types. The values in the table indicates the number of instances of a specific resource(for example:3321under the last column indicates that there are 3 A-type, 3 B-type, 2 C-type and 1 D-type resources are available after allocating the resources to all five processes). The numbers under allocation-column indicate that those number of resources are allocated to various processes mentioned in the first column. The numbers under Max- column indicate the maximum number of resources required by the processes. For example: in 1st row under allocation-column 2 0 0 1 indicate there are 2 A-type, 0 B-type, 0 C-type and 1 D- type resources are allocated to processP0. Whereas 4 2 1 2 under Max-column indicate that process P0’s maximum requirement is 4 A- type, 2 B-type,1C-type and 2 D-type resources.

Process	Allocation A B C D	Max ABC D	Available A B C D
P0	2 0 0 1	4 2 1 2	3 3 2 1
P1	3 1 2 1	5 2 5 2	
P2	2 1 0 3	2 3 1 6	
P3	1 3 1 2	1 4 2 4	
P4	1 4 3 2	3 6 6 5	

Answer the following questions using banker's algorithm by providing all intermediate steps

- a. How many instances of resources are present in the system under each type of a resource?
- b. Compute the Need matrix for the given snapshot of a system.
- c. Verify whether the snapshot of the present system is in a safe state by demonstrating an order in which the processes may complete. If a request from process P1 arrives for (1, 1, 0, 0), can the request be granted immediately?
- d. If a request from process P4 arrives for (0, 0, 2, 0), can the request be granted immediately?

Exercise6:

- a. Write a C program to simulate the following memory management technique: Paging

Exercise7:

- a. Write a C program that takes one or more file/directory names as command line input and reports following information
 - i) File Type ii) Number of Links iii) Time of last Access iv) Read, write and execute permissions.
- b. Write a C program to list every file in directory, its inode number and file name

TEXTBOOKS:

1. Operating System Principles - Abraham Silberchatz, Peter B. Galvin, Greg Gagne, JohnWiley, 9thEdition, 2018.
2. Advanced Programming in the UNIX Environment - W. Richard. Stevens, 3rd Edition, Pearson Education, NewDelhi, India. 2013.

REFERENCEBOOKS:

1. Internals and Design Principles-William Stallings,Operating Systems,9thEdition,PearsonEducation/PHI,2018.
2. Unix and shell Programming -BehrouzA.Forouzan,RichardF.Gilberg.Thomson
3. UNIXProgrammingEnvironment-KernighanandPike,PHI/PearsonEducation.2015.

Experiment 1:

PERIPHERALS OF A COMPUTER (INPUT/OUTPUT DEVICES)

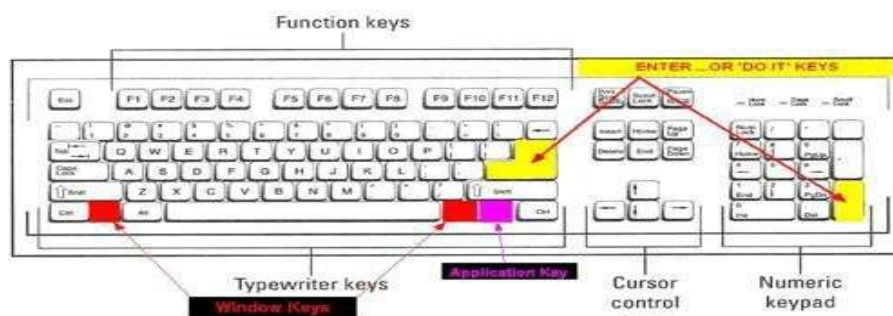
A computer peripheral refers to any external device that facilitates the interaction between a user and a computer system by either providing input or output functionalities. Input peripherals, like keyboards and mice, enable users to input data into the computer, while output peripherals, such as monitors and printers, display or produce information generated by the computer. Additionally, certain peripherals, like external hard drives, offer both input and output capabilities, enhancing the storage and transfer of data between the computer and external sources.

INPUT DEVICES:

These devices convert data into a binary format that the computer can understand and process. They transmit this binary data to the computer for further processing and also receive data from external sources. Common examples of such input devices include keyboards, mice, joysticks, trackballs, and scanners.

Keyboard:

The keyboard stands out as one of the most prevalent input devices, allowing users to manually input data through typing. Resembling a typewriter keyboard, it features additional specialized keys, such as function keys and control keys. These keys can be customized by users to suit their specific requirements, offering a personalized and adaptable interface for interacting with the computer system.



Mouse:

The mouse functions primarily as a pointing device, serving as an input tool for transmitting the coordinates of the screen's cursor position. Unlike keyboards that input characters, the mouse interacts with graphical user interface (GUI) software by relaying precise location data. This enables users to navigate interfaces, select items, and execute commands efficiently, particularly within GUI-based software applications.



Joystick And Tracker ball:

Joysticks and trackballs also function as pointing devices, akin to the mouse, serving a similar purpose in facilitating user interaction with graphical interfaces. These devices allow users to manipulate on-screen elements by providing directional control or precise cursor movement, offering alternative input options for navigating and interacting with software applications.



Scanner:

Scanners capture comprehensive image data directly from their source, typically a page, and store it in a graphic format suitable for display on screens. This process enables the reproduction of detailed visual content, facilitating the digitalization of physical documents or images for viewing and manipulation on electronic devices.



Touch Pads: Touch pads are tactile devices positioned on desktop surfaces, designed to detect and react to applied pressure.



Bar Code Readers:

Barcoded data finds common application in labeling goods and numbering books. A Barcode Reader is instrumental in this process, scanning the barcode image and converting it into an alphanumeric value. This value is then transmitted to the connected computer, facilitating efficient data entry and management.



Optical Mark Reader (OMR):

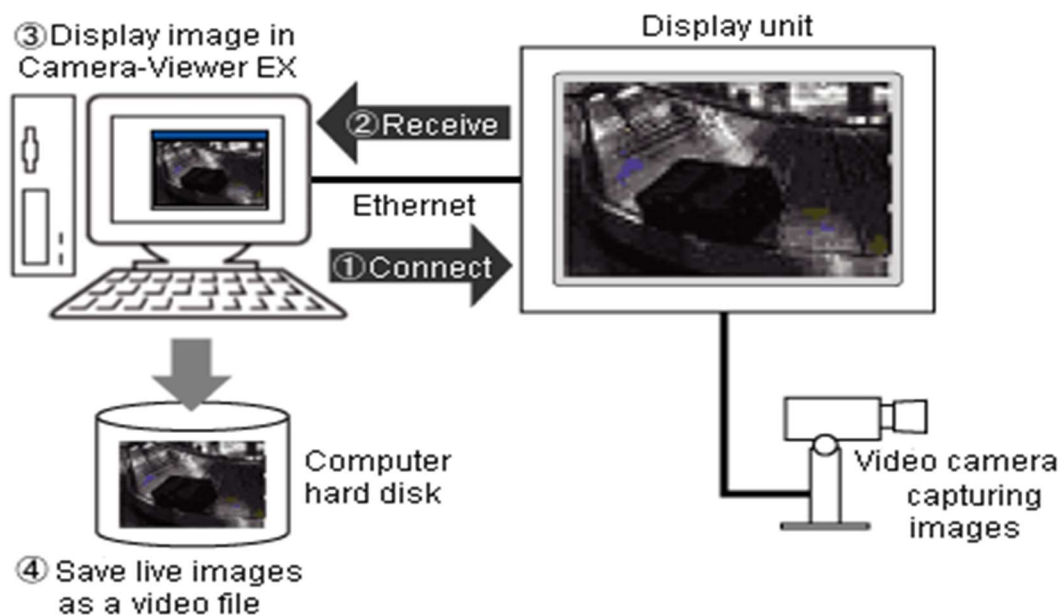
OMR (Optical Mark Recognition) technology is a specialized form of optical scanning utilized for identifying marks made by pens or pencils. It is particularly useful in scenarios where respondents need to select one option from a set of alternatives. OMR systems are commonly employed in educational settings to process examination answer sheets containing multiple-choice questions efficiently.

OUTPUT DEVICES:

Output devices receive binary data from the computer and translate it into a format understandable by humans. They then present this converted information to the external world. Common output devices include video display units, printers, and plotters, each serving distinct purposes in conveying digital data into readable or tangible formats for users.

Video display unit:

A video display unit, reminiscent of a television screen, serves as a visual interface connected to a computer system. This device, also known as a raster scan device, renders digital information into visible images, providing users with a graphical representation of data and facilitating interaction with the computer interface.



Printers:

There are several types of printers available, each designed for specific printing needs:

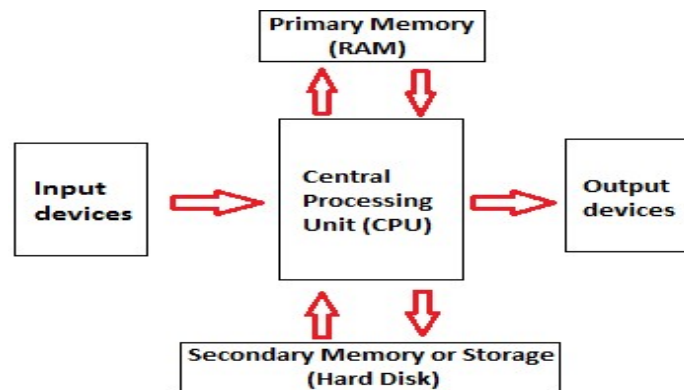
Character Printer: These printers operate by printing one character at a time, making them suitable for applications requiring limited text output.

Line Printer: Line printers are capable of printing one line of text at a time, offering faster printing speeds compared to character printers and suitable for moderate-volume printing tasks.

Page Printer: Page printers are designed to print an entire page at once, providing efficient and high-quality output ideal for documents, graphics, and images.



COMPONENTS OF CPU AND ITS FUNCTIONS



Every computer has three major components. they are:

- CPU.
- INPUTUNIT.
- OUTPUTUNIT

CENTRAL PROCESSING UNIT (CPU):

The Central Processing Unit (CPU), often referred to as the computer's brain, plays a pivotal role in controlling program execution, performing calculations, and overseeing various computer operations. It encompasses several key components, each serving distinct functions:

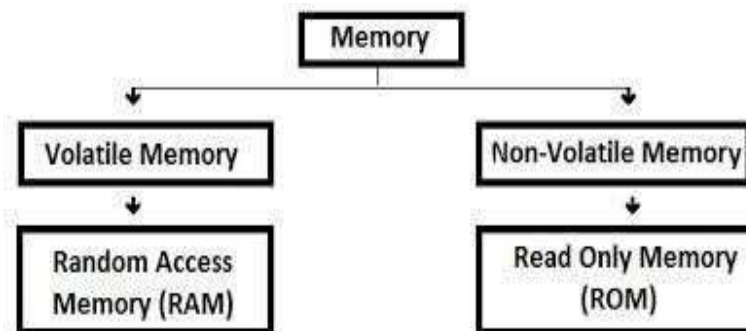
CPU Consist of following parts and Its Functions:

- a) MemoryUnit.
- b) Arithmetic and LogicUnit
- c) ControlUnit

a) MemoryUnit:

Memory serves as a fundamental component of computer systems, playing a crucial role in storing and retrieving data. It significantly influences the performance of the computer. Memory can be categorized into two main types:

- **Primary Memory/Volatile Memory:** Also known as primary storage, this type of memory is volatile, meaning it loses its stored data when power is turned off. It includes Random Access Memory (RAM), which is used by the computer to temporarily store data and program instructions during operation.
- **Secondary Memory/Non-Volatile Memory:** Secondary storage, or secondary memory, is non-volatile, meaning it retains stored data even when power is turned off. Examples include hard disk drives (HDDs), solid-state drives (SSDs), optical discs (such as CDs and DVDs), and USB flash drives. Secondary memory is used for long-term storage of data and programs.



b) Arithmetic and LogicUnit

The Arithmetic and Logic Unit (ALU) serves as the computational powerhouse within the CPU, responsible for executing a wide range of arithmetic and logical operations. These operations include:

Arithmetic Operations:

- Addition
- Subtraction
- Multiplication
- Division

Logical Operations:

- Comparison between numbers, letters, and special characters
- Logical AND
- Logical OR

Additionally, the ALU handles various conditional operations, including:

- Equal-to condition
- Less-than condition
- Greater-than condition

Overall, the ALU plays a critical role in processing data and executing complex calculations necessary for the functioning of the computer system.

c) **ControlUnit**

The Control Unit serves as the central coordinator within the computer system, directing and orchestrating the activities of other units. Its responsibilities include:

- Instructing the input unit on where to store data received from users.
- Regulating the flow of data and instructions between the storage unit and the Arithmetic and Logic Unit (ALU).
- Overseeing the transmission of results from the ALU back to the storage unit.

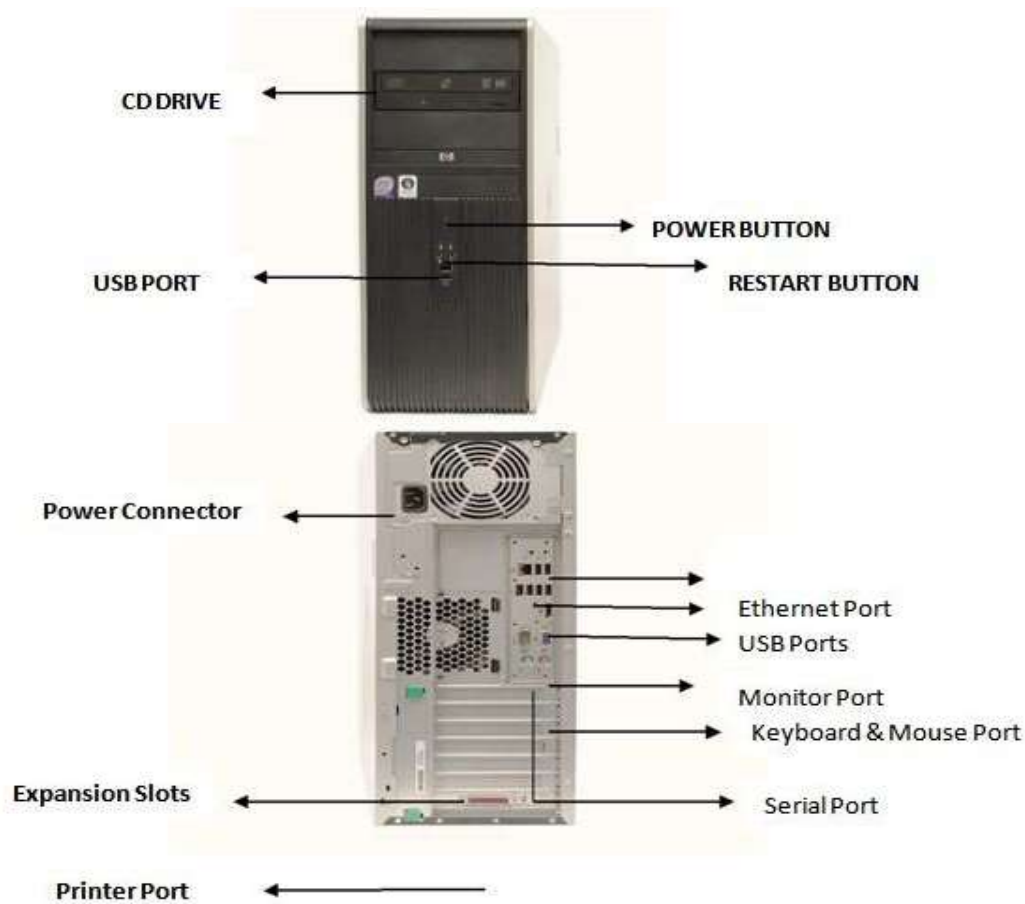
Often likened to the central nervous system of the computer, the Control Unit ensures the smooth and synchronized operation of the entire system, playing a vital role in managing the flow of information and facilitating efficient processing.

DISASSEMBLING AND ASSEMBLING THE PC

Before delving into the disassembling and assembling process, it's essential to understand several key components of a computer system: Cabinet, Mother Board, Processor, SMPS, RAM, HDD.

CABINET:

The cabinet serves as the protective outer shell of a computer system, safeguarding vital components such as the motherboard, processor, and others from environmental factors like dust, heat, and moisture. It acts as a shield, preserving the integrity and longevity of the internal hardware components, ensuring optimal performance and reliability.

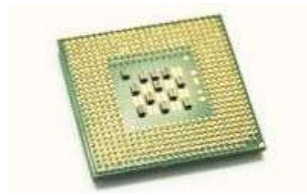


MOTHERBOARD



The motherboard serves as the central hub of a computer system, housing essential components such as the CPU, memory modules, connectors for storage drives, expansion slots for additional hardware, and ports for external devices like USB peripherals. It acts as a vital interface, directly or indirectly connecting every part of the computer, facilitating seamless communication and coordination between hardware components for efficient operation.

CPU/PROCESSOR



The central processing unit (CPU), or processor, resides within the computer case, typically situated on the motherboard. It appears as a compact ceramic square, housing a silicon chip approximately the size of a thumbnail. Inserted into the motherboard's CPU socket, the CPU operates alongside a heat sink, responsible for dissipating heat generated during operation.

The speed of a processor is gauged in megahertz (MHz), representing millions of instructions per second, or gigahertz (GHz), indicating billions of instructions per second. A higher processor speed denotes the capability to execute instructions more rapidly, enhancing overall system performance.

Various manufacturers produce processors for personal computers, with Intel and AMD being the most prominent names in the industry. These companies specialize in designing and manufacturing CPUs tailored to meet diverse computing needs and preferences.

POWER SUPPLY UNIT



The power supply unit (PSU) in a computer is responsible for transforming the electrical power from the wall outlet into the specific type of power required by the computer's components. It distributes this power through cables to various parts of the computer, including the motherboard and other internal components, ensuring they receive the necessary electrical supply to function properly.

RAM (RANDOM ACCESS MEMORY)



RAM, or Random Access Memory, acts as the short-term memory of your computer system. When your computer executes calculations or processes data, it temporarily stores this information in the RAM until it's required for ongoing tasks or operations.

HARD DRIVE



The hard drive in your computer serves as the primary storage location for software installation, as well as storing your documents and other files. It functions as long-term storage, retaining data even when the computer is powered off or disconnected. When you initiate a program or access a file, the computer retrieves relevant data from the hard drive and temporarily stores it in the RAM for quick access during ongoing tasks. Similarly, when you save a file, the data is copied back to the hard drive for permanent storage. The speed of the hard drive directly impacts the computer's performance, influencing factors such as startup time and program loading speed.

STEPS FOR DISASSEMBLING:

Step 1: Unplugging: Disconnect all cables from the computer, including power, USB, mouse, keyboard, internet, and modem.

Step 2: Removing Outer Shell/Casing: Unscrew and remove the side panels of the computer case by sliding them off.

Step 3: Removing Top Panel and Front Panel: Slide off the top panel of the computer case. Remove the front panel by pushing in tabs and sliding it forward.

Step 4: Removing System Fan: Unplug the system fan from the motherboard. Unscrew and remove the system fan from the back of the computer.

Step 5: Removing CPU Fan: Unplug the CPU fan from the motherboard. Unscrew and remove the CPU fan from the top of the CPU heat sink.

Step 6: Removing Power Supply (SMPS): Unplug all wires connected to the power supply. Unscrew and remove the power supply from the back of the computer.

Step 7: Removing CD/DVD Drive(s): Unplug the ribbon cable from the back of the drive. Slide out the CD/DVD drive from the inside of the computer.

Step 8: Removing Hard Disk Drive: Disconnect cables from the hard drive. Unscrew and remove the hard drive from its slot.

Step 9: Removing Expansion Cards: Remove screws from occupied expansion card slots. Carefully pull upward to remove expansion cards.

Step 10: Removing Connectivity Centre Cables (front and back panel): Unplug all cables from the connectivity center. Unplug wires from power button, HDD light, and power light.

Step 11: Removing RAM (Random Access Memory): Push down on tabs at both ends of RAM to release it.

Step 12: Removing Power Button; Power LED + HDD LED: Cut zip tie holding wires for front connectivity center. Press tabs on chassis' side to remove it. Push LEDs from front and button from back to remove.

Step 13: Removing Motherboard: Remove screws holding motherboard to frame. Lift motherboard out of the frame.

Step 14: Done!



Case:

- Metal Right Panel, Left Panel
- Plastic Front Panel, Top Panel
- Metal & Plastic Frame

Hardware:

- Hard Drive
- CD/DVD Drive
- Power Supply
- Expansion Cards
- RAM Chips
- Connectivity Center Cables/Wires:
 - SATA Cable (Hard Drive to Motherboard)
 - Portable HDD Dock and Wires (Power and Data from Portable HDD to Motherboard)
 - Drive Ribbon (CD/DVD Drive to Motherboard)

Miscellaneous:

- 33 Screws
- Drive Slot Cover
- 2 Expansion Card Slot Covers
- Plastic Piece (referred to as "chassis" for a button and LEDs)
- Large System Fan
- Small CPU Fan
- Portable HDD Slot

STEPS FOR ASSEMBLING

- a. Get Ready: Find a clean, well-lit area and gather all your tools and computer parts.
- b. Put in the Power Supply: Slide the power supply into the case and screw it in place.
- c. Attach the Motherboard: Put the motherboard inside the case and screw it onto the raised spots.
- d. Install the Processor (CPU): Open the CPU socket, put the CPU in correctly, and close the socket.
- e. Attach the CPU Cooler: Apply thermal paste, attach the cooler to the CPU, and secure it.
- f. Add Memory (RAM): Open the slots, put the RAM sticks in carefully, and close the slots.
- g. Install Storage Drives: Slide the hard drive or SSD into its slot and connect the cables.
- h. Connect Front Panel Wires: Attach the case's power button, lights, and other wires to the motherboard.
- i. Put in Expansion Cards: Insert any extra cards like graphics cards and secure them.
- j. Connect Case Fans: Put in any fans and connect them to the motherboard.
- k. Organize Cables: Keep the cables tidy by bundling them together with ties.
- l. Check Everything: Double-check all connections and make sure everything is in place.
- m. Close the Case: Put the side panels back on and screw them shut.
- n. Turn It On: Plug in the computer, press the power button, and make sure it starts up.
- o. Install Software: Insert the operating system disk and follow the instructions to install it.
- p. Install Drivers: Install any extra software needed for your hardware to work properly.

WINDOWS INSTALLATION

Windows 10, part of the Microsoft Windows operating system family, is the latest version designed for desktop computers. It was released to the public on July 15, 2015. There are two main editions: Windows 10 Home, for personal use, and Windows 10 Professional, offering extra features like dual-processor support and domain joining for centrally managed networks.

Installation Process:

Installing Windows 10 typically takes about 1 hour and 30 minutes. To simplify the process, it's broken down into steps. Here's how to begin:

System Requirements:

1. **Processor:** 1GHz or faster.
2. **RAM:** 1GB for 32-bit OS, 2GB for 64-bit OS.
3. **Storage:** 16GB for 32-bit OS, 20GB for 64-bit OS.
4. **Graphics Card:** DirectX 9 with WDDM driver.
5. **Display:** 800x600 resolution.
6. **Network:** Internet connection.

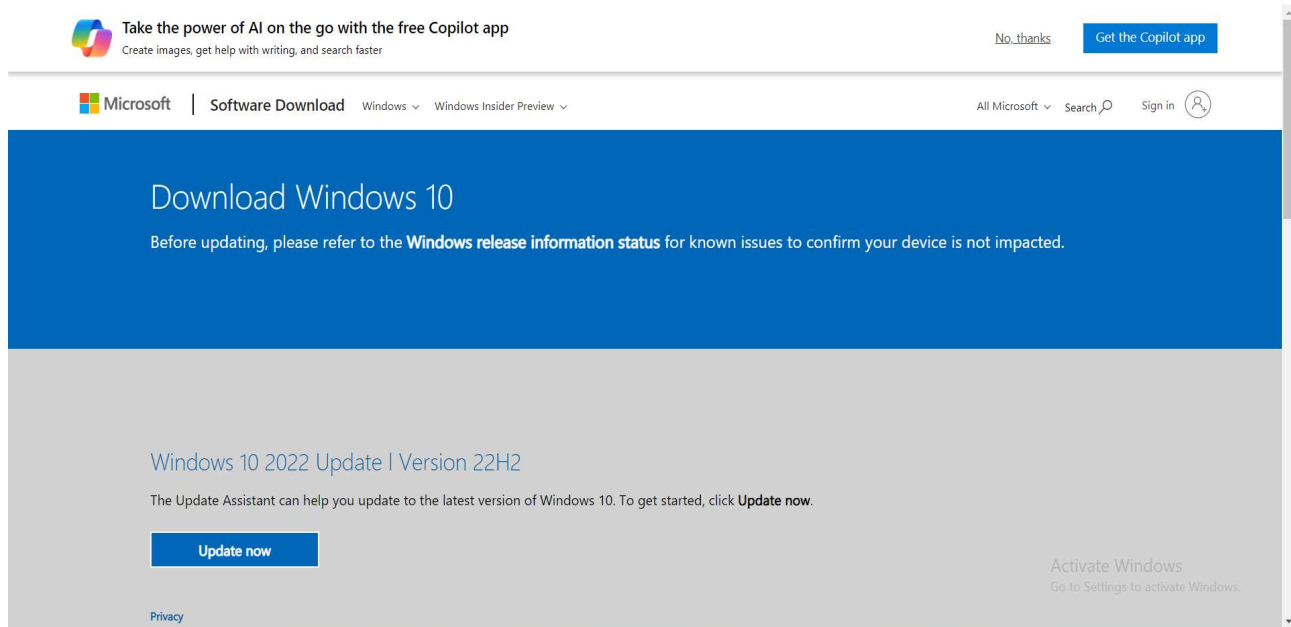
Installing Windows 10

There are two main methods:

- Full installation and
- Upgrading from an older version.

Full Installation

1. **Download Windows 10 Media:** Go to the Microsoft website and download the installation media. <https://www.microsoft.com/en-gb/software-download/windows10>

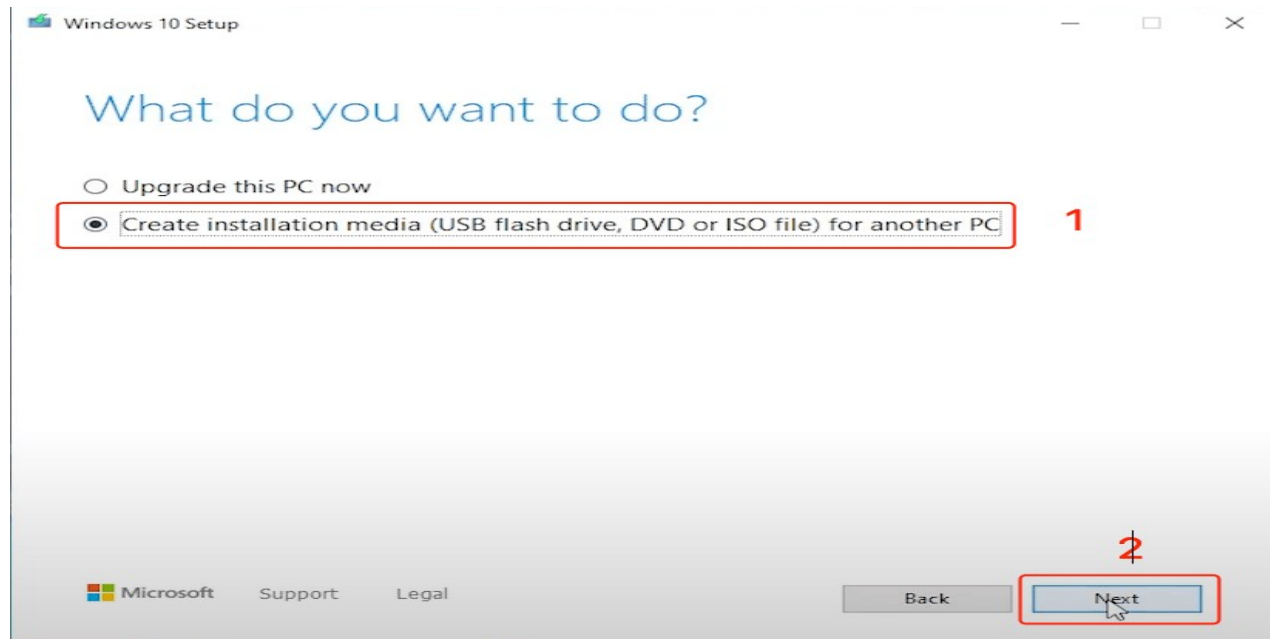


2. **Accept License Terms:** Follow the prompts to accept the license terms.



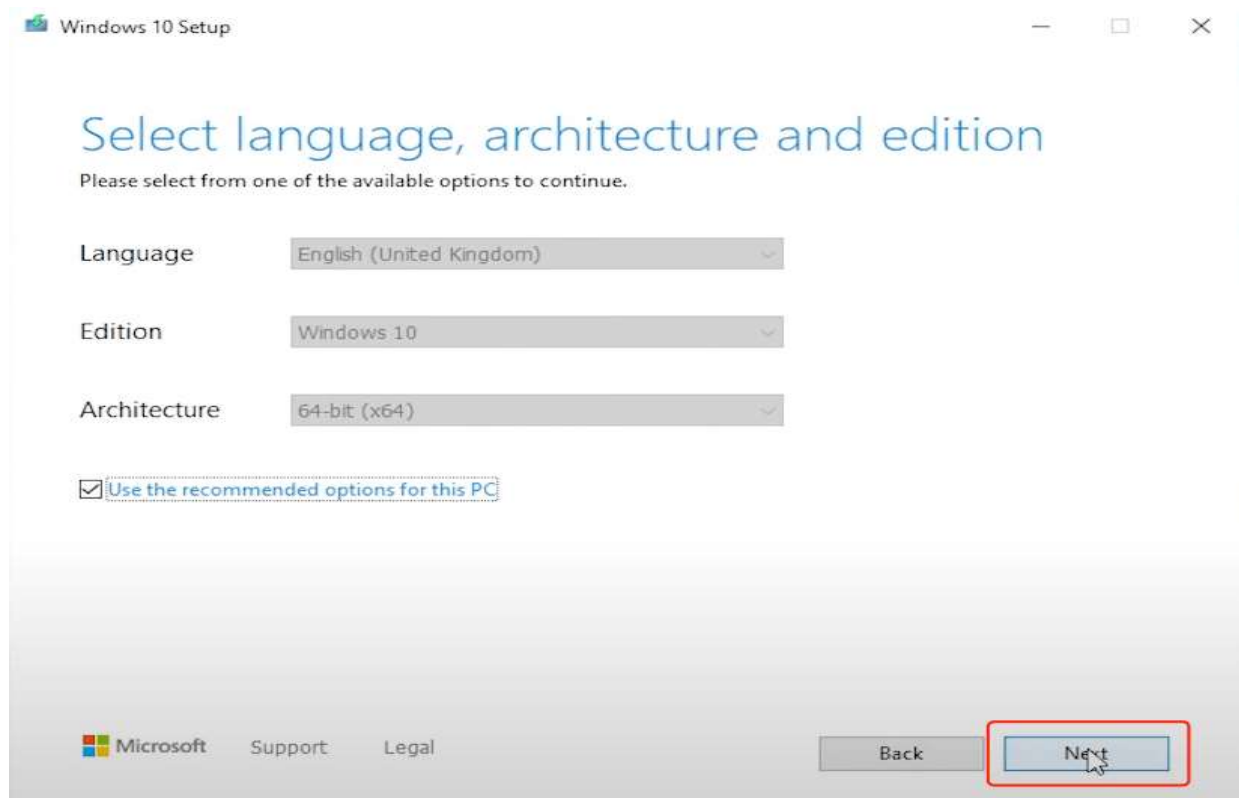
3. **Create Installation Media:**

- Choose to create installation media.
- Select USB flash drive as the media type.
- Plug in an 8GB or larger USB drive.
- Follow the prompts to download and create the bootable USB drive.



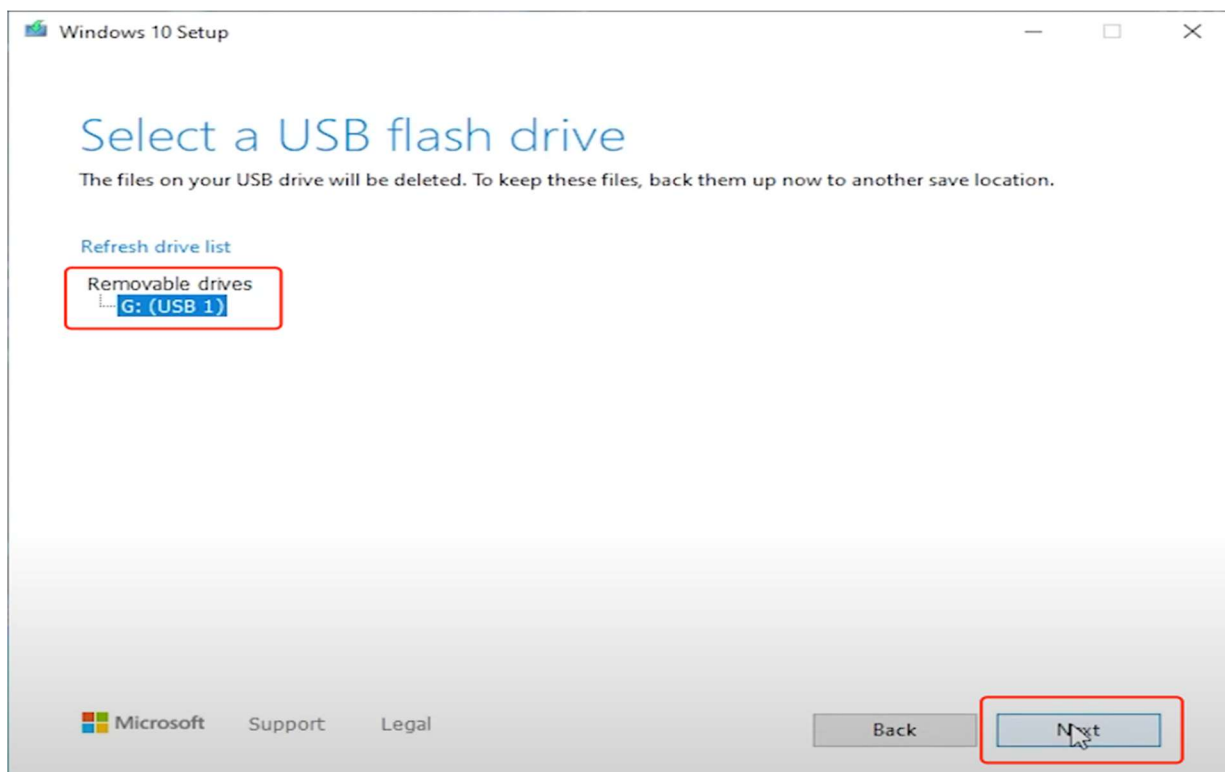
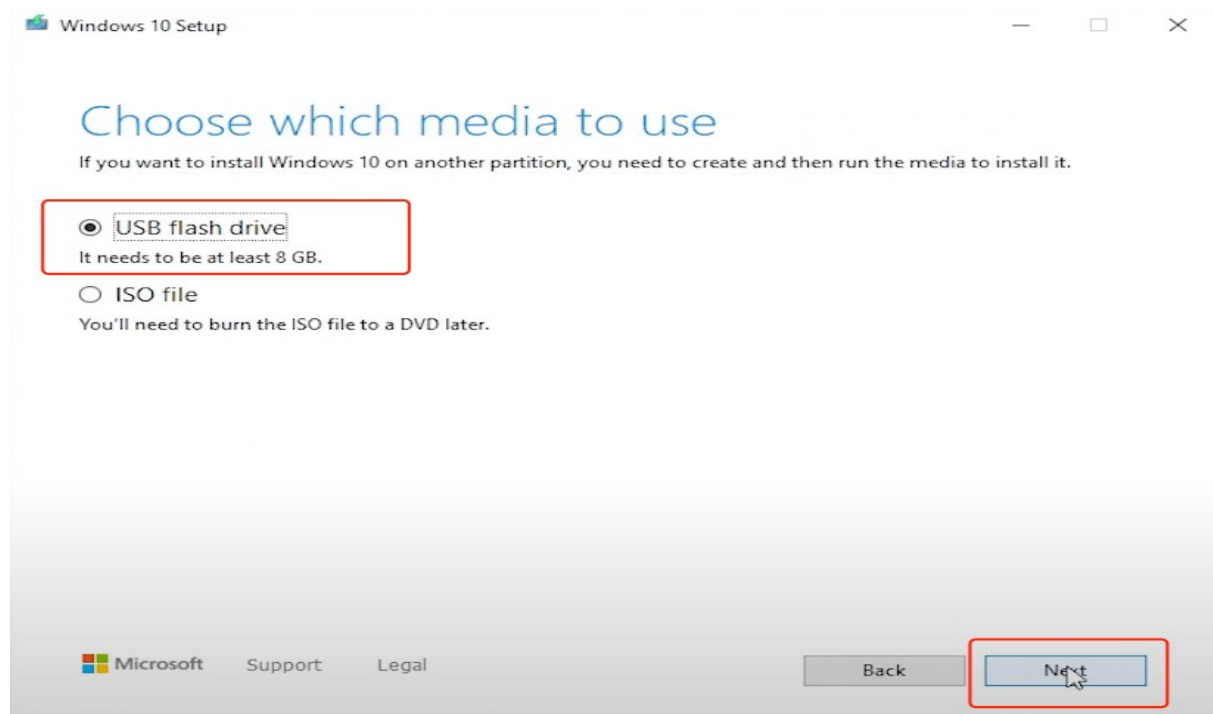
4. **Boot from USB:**

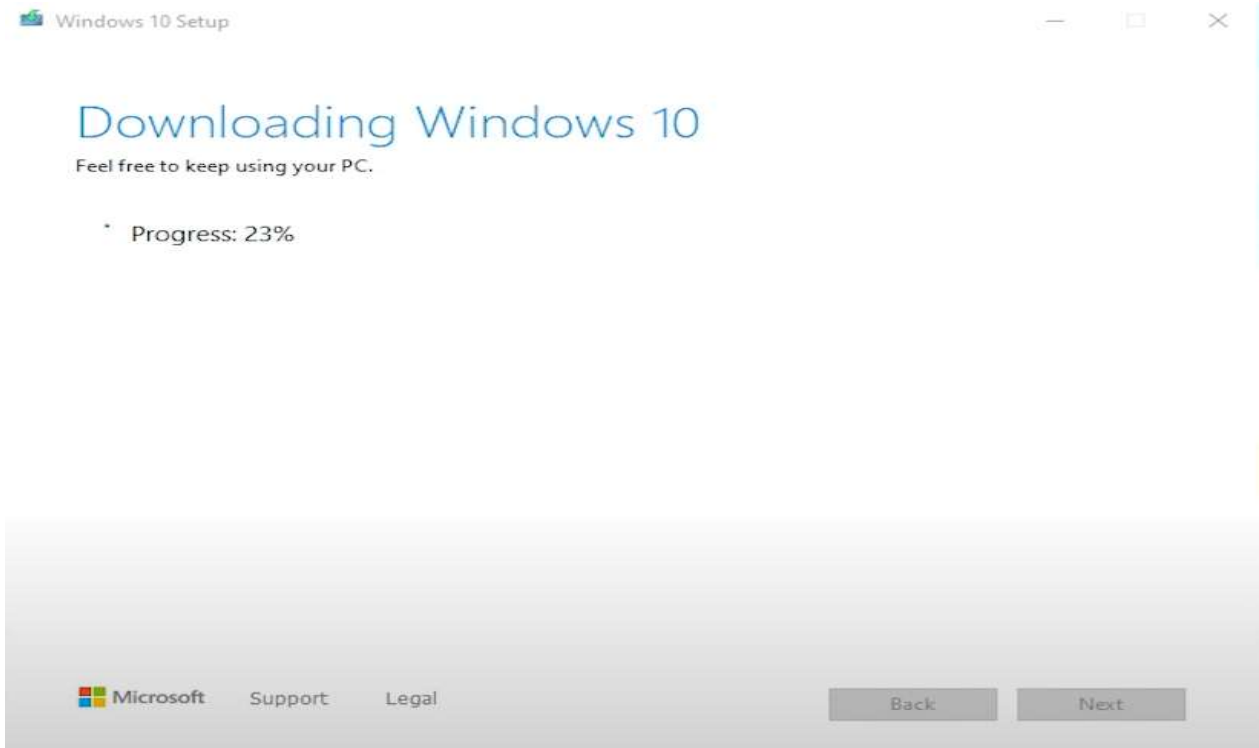
- Insert the USB drive into your PC.
- Restart your PC and enter BIOS/UEFI (usually by pressing F2, F10, or Del).
- Set the USB drive as the first boot option.
- Save changes and exit BIOS/UEFI.



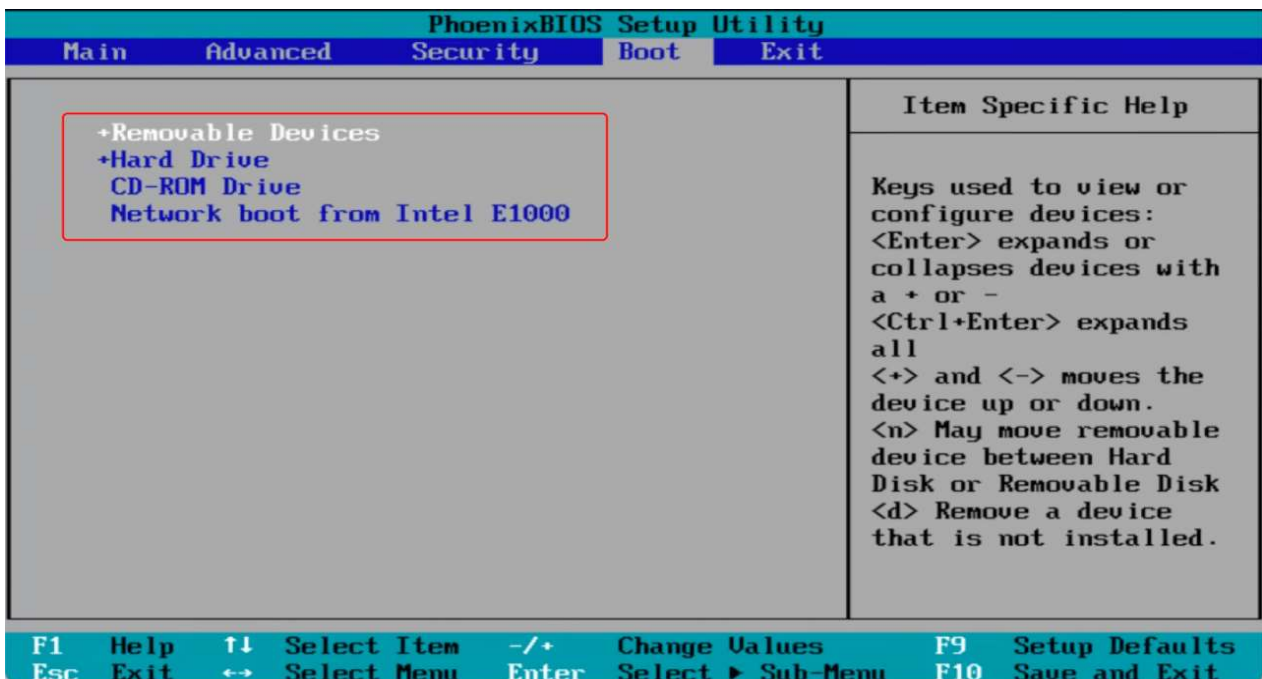
5. Install Windows 10:

- When the Windows Setup screen appears, select your language and other preferences.
- Click "Install Now".
- Follow the prompts to complete the installation.





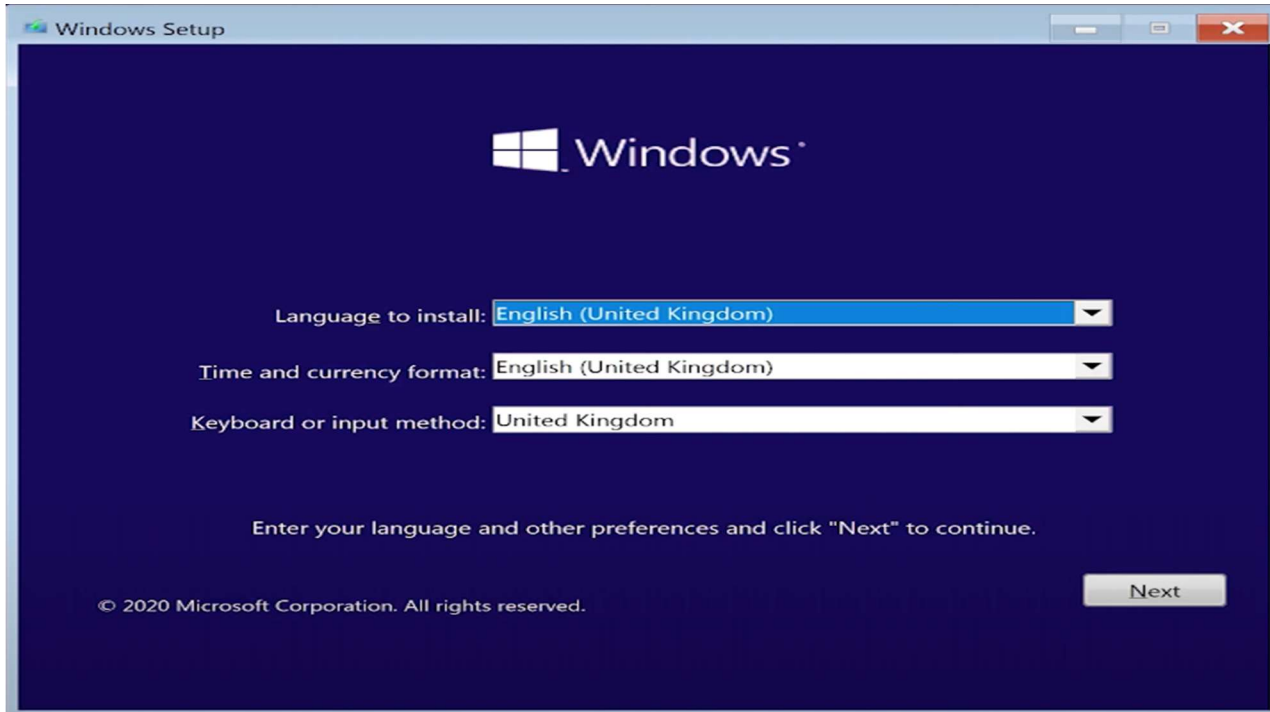
6. **Plug in USB:** Insert the USB drive with Windows 10 installer into your new PC.
7. **Turn On PC:** Start your computer and enter the BIOS/UEFI by pressing F2, F10, or Del.



8. **Set Boot Order:** In BIOS/UEFI, set the USB flash drive as the first boot option.
9. **Save and Exit:** Save the changes and exit. Your PC will restart and boot from the USB drive.

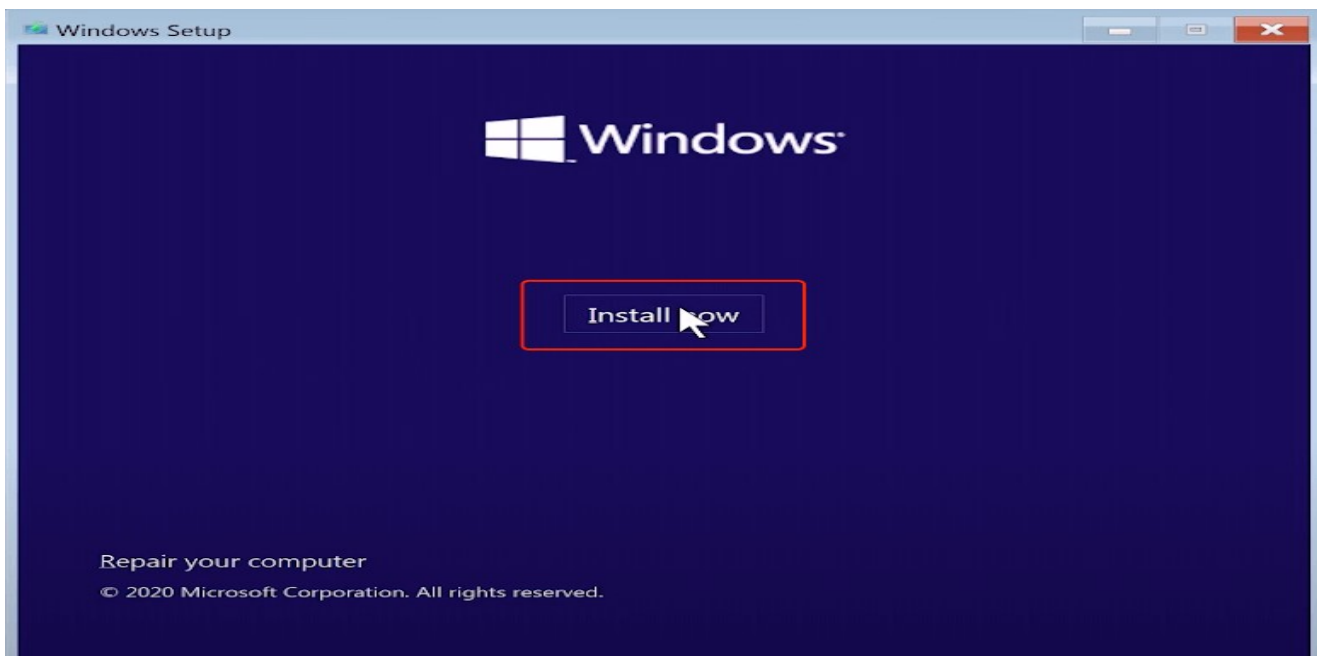
10. Windows Setup:

- When the setup screen appears, choose your language, time, and currency format.
- Click "Next".



11. Install Windows:

- Click "Install Now".
- Follow the prompts to complete the Windows installation.



Ubuntu (LINUX) Installation

Ubuntu is a Linux operating system and distribution designed for personal computers, smartphones, and network servers. It utilizes the Unity desktop environment by default and is built upon Debian-based architecture. Ubuntu is renowned for its commitment to free software and is named after the Southern African philosophy of ubuntu, emphasizing the interconnectedness and shared humanity among individuals.

Step 0: Prerequisites

1. **Power Source:** Keep your laptop plugged in.
2. **Free Storage:** Ensure you have at least 25 GB free space (5 GB for minimal installation).
3. **Backup:** Backup your data to prevent any loss.
4. **USB/DVD:** Have an 8GB+ USB or DVD with the Ubuntu version you want.

Step 1: Download Ubuntu

1. **Get Ubuntu:** Download the Ubuntu image from the Ubuntu release page

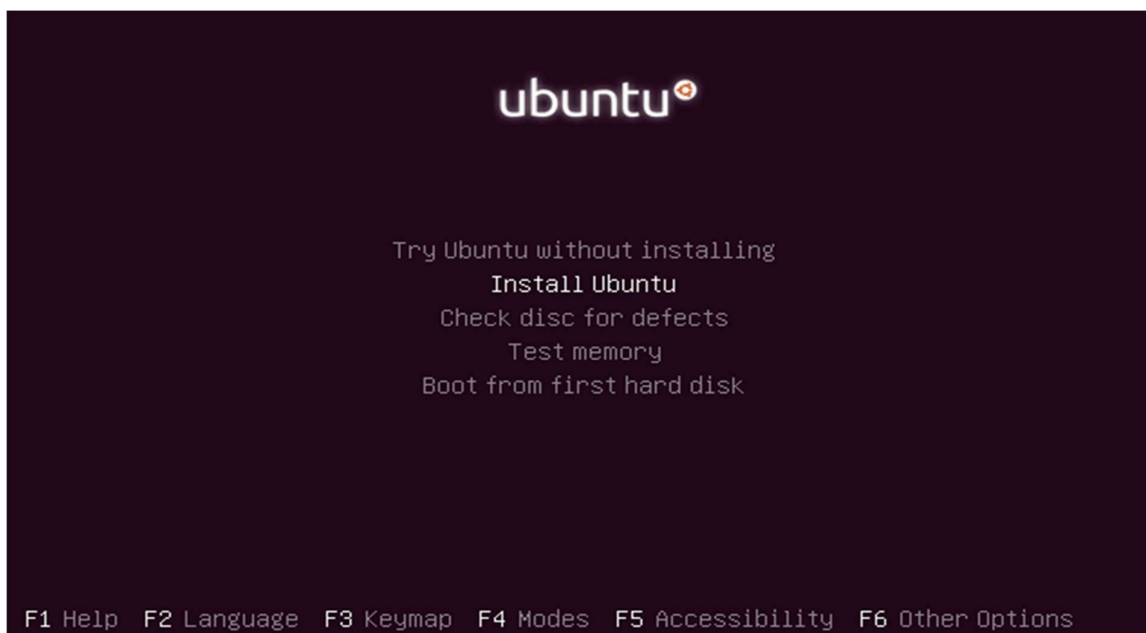
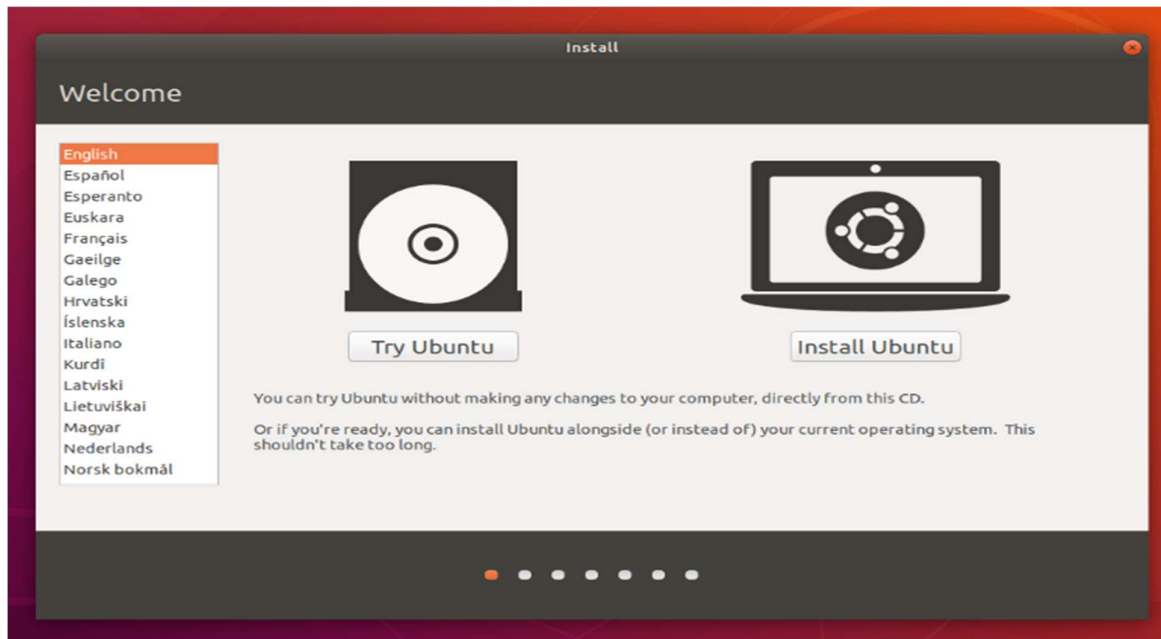
<https://releases.ubuntu.com/>



2. **Get UNetbootin:** Download and install UNetbootin <https://unetbootin.github.io/>
3. **Burn Image:** Use UNetbootin to burn the Ubuntu image to your USB stick.

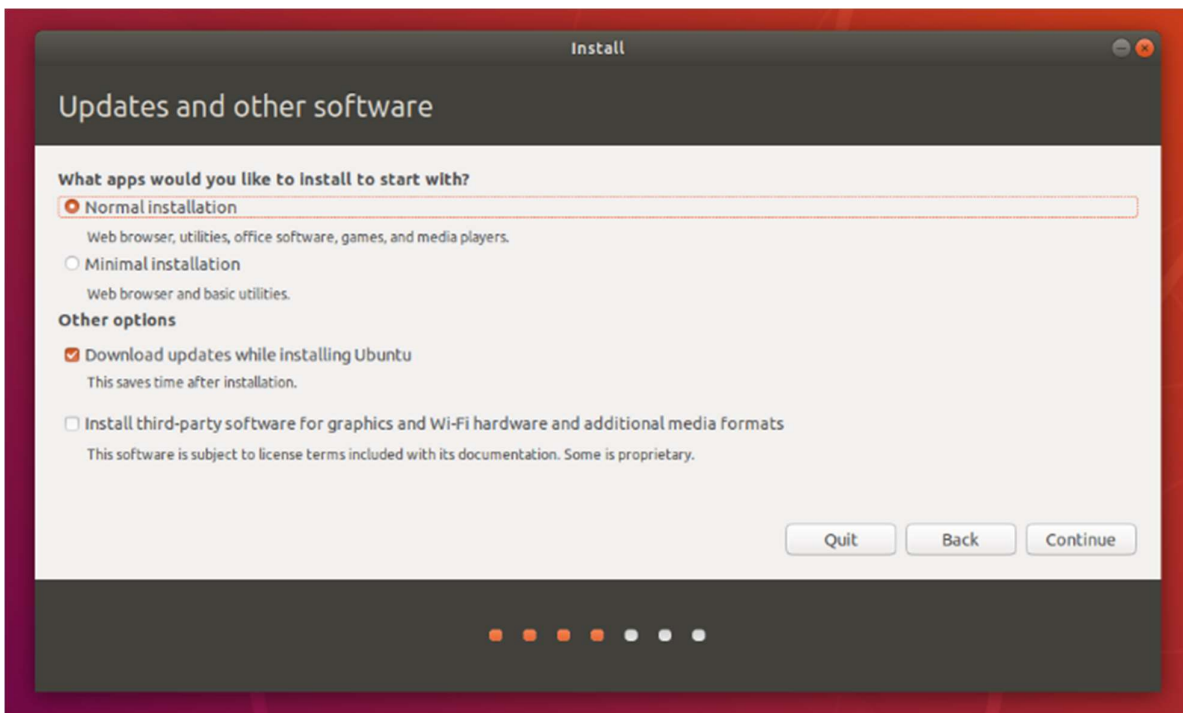
Step 2: Boot from USB/DVD

1. **Insert Media:** Put the Ubuntu DVD in your drive or plug in the USB stick.
2. **Restart Computer:** Restart and a welcome screen will appear.
3. **Select Language:** Choose your language and decide if you want to try or install Ubuntu.
4. **Install Ubuntu:** Click "Install Ubuntu" to start the installation.



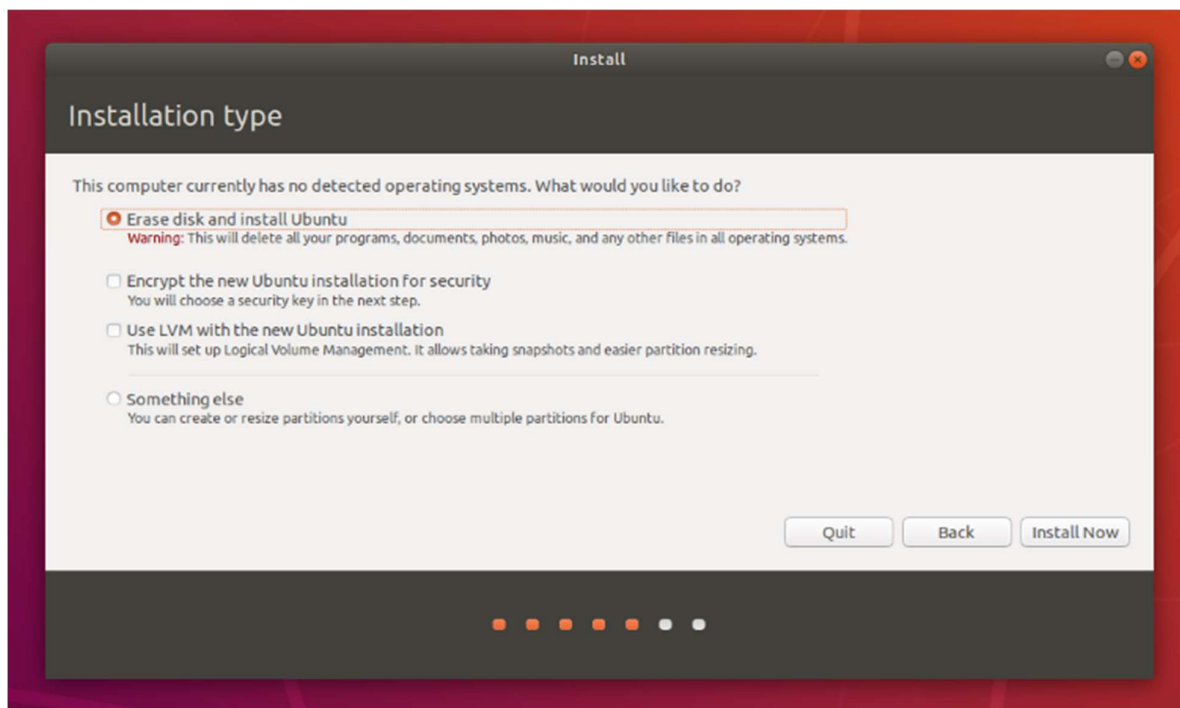
Step 3: Preparation

1. **Keyboard Layout:** Choose your keyboard layout.
2. **Installation Type:** Select "Normal installation" or "Minimal installation".
3. **Third-Party Software:** Check the box to download updates and third-party software.



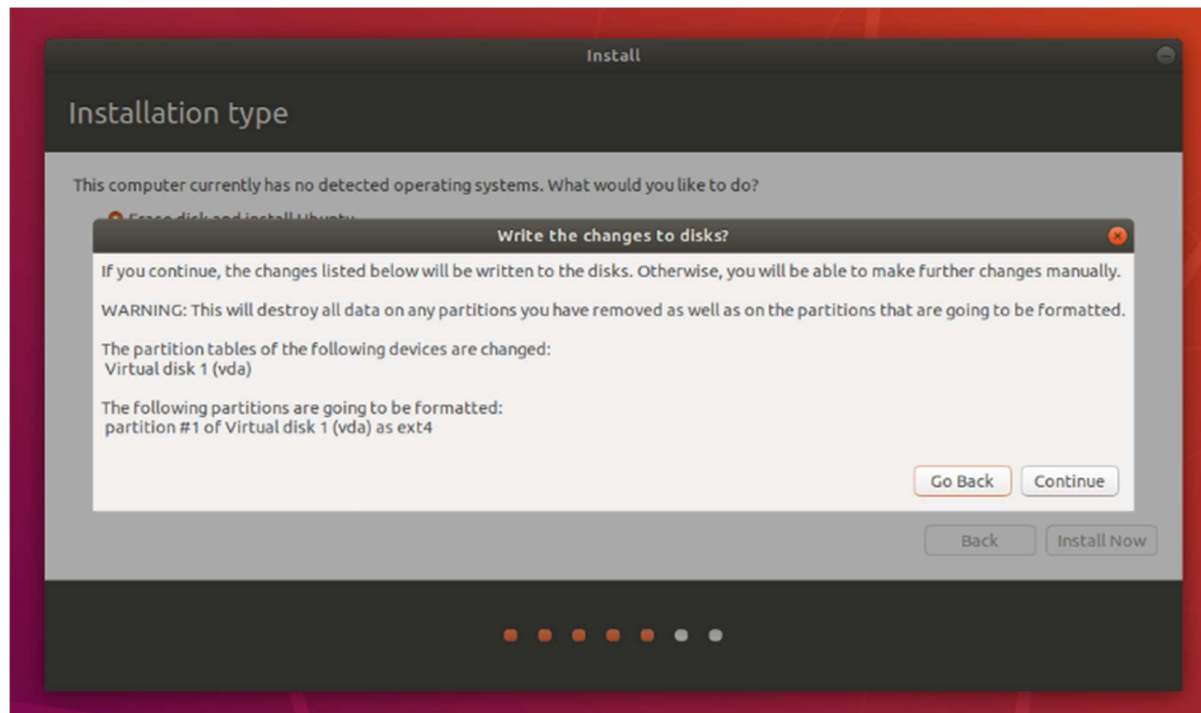
Step 4: Allocate Drive Space

1. **Choose Option:** Decide to install Ubuntu alongside another OS, replace the existing OS, or choose "Something else" for custom partitioning.
2. **Install Now:** Click "Install Now".



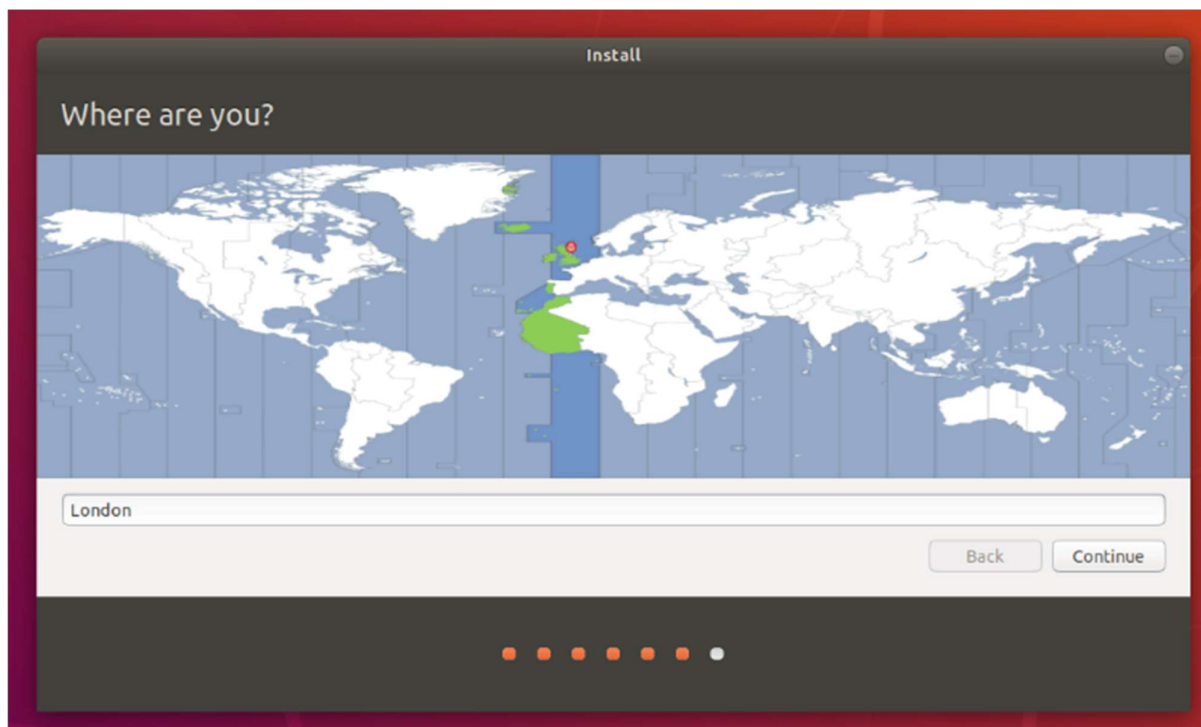
Step 5: Begin Installation

1. **Review Changes:** Confirm your choices and click "Continue".



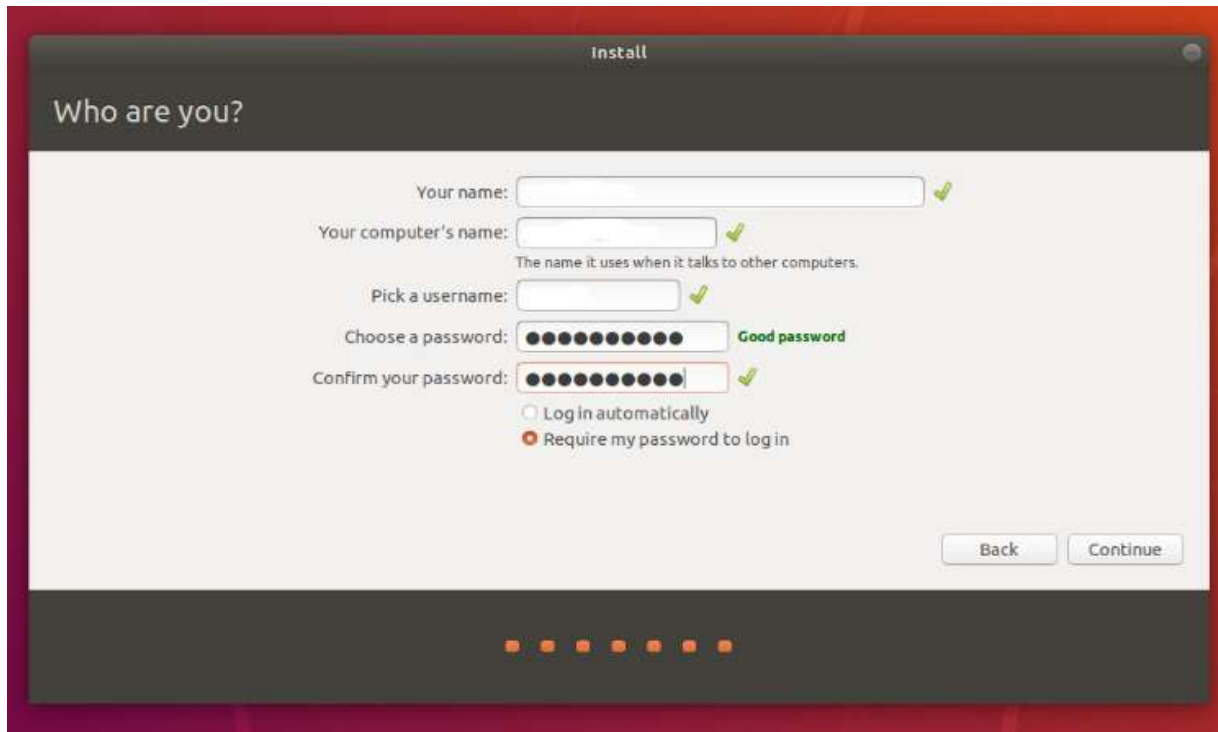
Step 6: Select Location

1. **Detect Location:** Ensure your location is correct and click "Forward".



Step 7: Login

1. **Create Account:** Enter your name, computer name, username, and a strong password.
2. **Security Options:** Choose to enable automatic login and/or home folder encryption.



The image shows the 'Who are you?' screen in the Ubuntu installer. The window title is 'Install'. The title bar is dark grey. The main content area is light grey. At the top, it says 'Who are you?'. Below this, there are several input fields with green checkmarks indicating successful validation:

- Your name: [text input] ✓
- Your computer's name: [text input] ✓
The name it uses when it talks to other computers.
- Pick a username: [text input] ✓
- Choose a password: [password input] ✓ Good password
- Confirm your password: [password input] ✓

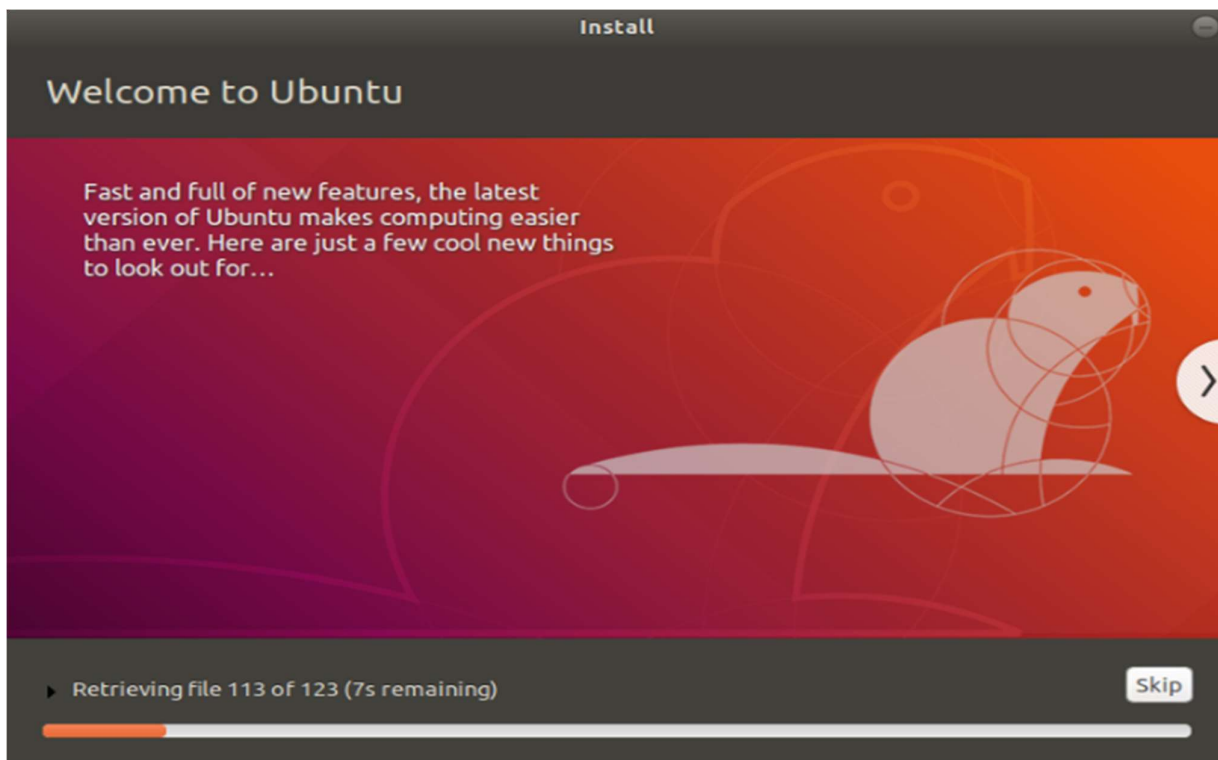
Below the password fields, there are two radio buttons:

- ☐ Log in automatically
- ☒ Require my password to log in

At the bottom right, there are two buttons: 'Back' and 'Continue'. At the bottom center, there is a progress bar with seven orange squares, the fourth of which is filled.

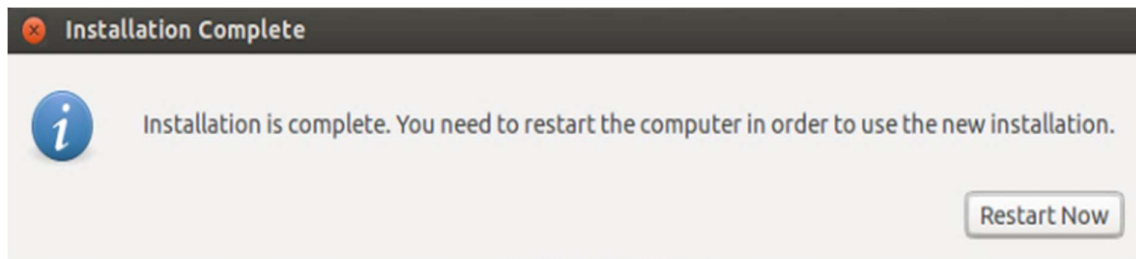
Step 8: Background Installation

1. **Installation Process:** Ubuntu will install while showing some information about its features.



Step 9: Complete Installation

1. **Finish:** Once done, a window will ask you to restart. Remove the USB/DVD and click "Restart Now".



Experiment 2

a) Implement in c language the following UNIX commands using systemcalls.

i) cat ii)ls iii)mv

i)Cat

PROGRAM:

```
#include<stdio.h>
#include<fcntl.h>
int main(int argc, char *argv[])
{
    char ch;
    int fd;
    fd=open(argv[1],O_RDONLY);
    if(fd==-1)
    {
        printf("File not opened");
    }
    printf("no of arguments %d",argc);
    while(read(fd,&ch,1))
    putchar(ch);
    close(fd);
    return 0;
```

ii)ls

PROGRAM:

```
#include<stdio.h>
#include<dirent.h>
int main(int argc, char *argv[])
{
    DIR *dirop;
    struct dirent *dired;
    if(argc!=2)
    {
        printf("Invalid no of argument");
    }
    if((dirop=opendir(argv[1]))==NULL)
        printf("cannot open the directory");
    while((dired=readdir(dirop))!=NULL)
        printf("%ld %s\n",dired->d_ino,dired->d_name);
    closedir(dirop);
}
```

iii)mv

PROGRAM:

```
#include<stdio.h>
#include<string.h>
int main(int argc, char *argv[])
{
    int r;
    if(argc<3)
        printf("wrong arguments");
    else
        if(argc==3)
        {
            printf("\n%s %s\n",argv[1],argv[2]);
            r=link(argv[1],argv[2]);
            printf("%d\n",r);
            unlink(argv[1]);
        }
    return 0;
}
```

b)Write a C program to create child process and allow parent process to display “parent” and the child to display “child” on the screen

PROGRAM:

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
#include <sys/types.h>
int main()
{
    pid_t ret_value;
    printf("\nThe process id is %d\n",getpid());
    ret_value = fork();
    if(ret_value< 0)
    {
        //fork has failed
        printf("\nFork Failure\n");
    }
    else if(ret_value == 0)
    {
        //child process
        printf("\nChild Process\n");
        printf("The process id is %d\n",getpid());
        sleep(20);
    }
    else
    {
        //parent process
        wait(NULL);
        printf("Parent Process\n");
        printf("The process id is %d\n",getpid());
        sleep(30);
    }
    return 0;
}
```

Experiment 3

Assume you have the following jobs to execute with one processor, with the jobs arriving in the order listed here:

i	T(pi)
0	80
1	20
2	10
3	20
4	50

With the following values write a program to get the required output which is listed below

- Suppose a system uses FCFS scheduling. Create a Gantt chart illustrating the execution of these processes?
- What is the turnaround time for process p3?
- What is the average wait time for the processes?

PROGRAM:

```
#include<stdio.h>
int main(){

    int bt[10]={0},at[10]={0},tat[10]={0},wt[10]={0},ct[10]={0};
    int n,sum=0;
    float totalTAT=0,totalWT=0;

    printf("Enter number of processes   ");
    scanf("%d",&n);

    printf("Enter arrival time and burst time for each process\n\n");

    for(int i=0;i<n;i++)
    {

        printf("Arrival time of process[%d]   ",i+1);
        scanf("%d",&at[i]);

        printf("Burst time of process[%d]     ",i+1);
        scanf("%d",&bt[i]);

        printf("\n");
    }

    //calculate finish time of processes

    for(int j=0;j<n;j++)
    {
        sum+=bt[j];
        ct[j]+=sum;
    }

    //calculate turnaround time and waiting times
```

```

for(int k=0;k<n;k++)
{
    tat[k]=ct[k]-at[k];
    totalTAT+=tat[k];
}

for(int k=0;k<n;k++)
{
    wt[k]=tat[k]-bt[k];
    totalWT+=wt[k];
}

printf("Solution: \n\n");
printf("JOBNO\t AT\t BT\t FT\t TAT\t WT\t\n\n");

for(int i=0;i<n;i++)
{
    printf("P%d\t %d\t %d\t %d\t %d\t %d\n",i+1,at[i],bt[i],ct[i],tat[i],wt[i]);
}

printf("\n\nAverage Turnaround Time = %f\n",totalTAT/n);
printf("Average WT = %f\n\n",totalWT/n);

return 0;
}

```

Experiment 4

a) Write a C program that illustrate communication between two unrelated process using named pipes

PROGRAM:

Mkfifo.c → To create a named pipe

```
#include<stdio.h>
#include<sys/types.h>
#include<sys/stat.h>
int main()
{
    int res;
    res = mkfifo("fifo1",0777); //creates a named pipe with the name fifo1
    printf("named pipe created\n");
}
```

Sender.c

```
#include <unistd.h>
#include <stdio.h>
#include <fcntl.h>
int main(){
    int res,n;
    res=open("fifo1",O_WRONLY);
    write(res,"Message",7);
    printf("Sender Process having PID %d, sent the data\n",getpid());
}
```

Receiver.c

```
#include <unistd.h>
#include <stdio.h>
#include <fcntl.h>
int main(){
    int res,n;
    char buffer[100];
    res=open("fifo1",O_RDONLY);
    n=read(res,buffer,100);
    printf("Reader process having PID %d started\n",getpid());
    printf("Data received by reciever %d is : %s\n",getpid(),buffer);
}
```

b) Write a C program that receives a message from message queue and display them.

PROGRAM:

Message Queue Sender:

```
#include<stdlib.h>
#include<stdio.h>
#include<string.h>
#include<unistd.h>
#include<sys/types.h>
#include<sys/ipc.h>
#include<sys/msg.h>
#define MAX_TEXT 512 //maximum length of the message that can be sent allowed
struct my_msg{
    long int msg_type;
    char some_text[MAX_TEXT];
};
int main()
{
    int running=1;
    int msgid;
    struct my_msgsome_data;
    char buffer[50]; //array to store user input
    msgid=msgget((key_t)12345,0666|IPC_CREAT);
    if (msgid == -1) // -1 means the message queue is not created
    {
        printf("Error in creating queue\n");
        exit(0);
    }

    while(running)
    {
        printf("Enter some text:\n");
        fgets(buffer,50,stdin);
        some_data.msg_type=1;
        strcpy(some_data.some_text,buffer);
        if(msgsnd(msgid,(void *)&some_data, MAX_TEXT,0)==-1) // msgsnd returns -1 if the message is not sent
        {
            printf("Msg not sent\n");
        }
    }
}
```



```

    }
    if(strncmp(buffer,"end",3)==0)
    {
        running=0;
    }
}
}

```

Message Queue Receiver:

```

#include<stdlib.h>
#include<stdio.h>
#include<string.h>
#include<unistd.h>
#include<sys/types.h>
#include<sys/ipc.h>
#include<sys/msg.h>
struct my_msg{
    long int msg_type;
    char some_text[BUFSIZ];
};
int main()
{
    int running=1;
    int msgid;
    struct my_msg some_data;
    long int msg_to_rec=0;
    msgid=msgget((key_t)12345,0666|IPC_CREAT);
    while(running)
    {
        msgrcv(msgid,(void *)&some_data,BUFSIZ,msg_to_rec,0);
        printf("Data received: %s\n",some_data.some_text);
        if(strncmp(some_data.some_text,"end",3)==0)
        {
            running=0;
        }
    }
    msgctl(msgid,IPC_RMID,0);
}

```

c) Write a C program to allow cooperating process to lock a resource for exclusive use (using semaphore)

PROGRAM:

```
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>
#include <semaphore.h>
#include <unistd.h>

void *fun1();
void *fun2();

int shared=1; //shared variable
sem_t s; //semaphore variable

int main()
{
    sem_init(&s,0,1); //initialize semaphore variable - 1st argument is address of variable, 2nd is number
of processes sharing semaphore, 3rd argument is the initial value of semaphore variable
    pthread_t thread1, thread2;
    pthread_create(&thread1, NULL, fun1, NULL);
    pthread_create(&thread2, NULL, fun2, NULL);
    pthread_join(thread1, NULL);
    pthread_join(thread2, NULL);
    printf("\nFinal value of shared is %d\n", shared); //prints the last updated value of shared variable
}

void *fun1()
{
    int x;
    printf("\nWe are in func1()\n");
    sem_wait(&s); //executes wait operation on s
    x=shared; //thread1 reads value of shared variable
    printf("\nThread1 reads the value as %d\n", x);
    x++; //thread1 increments its value
    printf("Local updation by Thread1: %d\n", x);
    sleep(1); //thread1 is preempted by thread 2
    shared=x; //thread one updates the value of shared variable
    printf("\nValue of shared variable updated by Thread1 is: %d\n", shared);
```

```
        sem_post(&s);
    }

void *fun2()
{
    int y;
    printf("\nWe are in func2()\n");
    printf("\nfun2() is blocked until func1() executes");
    sem_wait(&s);
    y=shared;//thread2 reads value of shared variable
    printf("\nFunc2() resumes its execution after func1()");
    printf("\nThread2 reads the value as %d\n",y);
    y--; //thread2 increments its value
    printf("\nLocalupdation by Thread2: %d\n",y);
    sleep(1); //thread2 is preempted by thread 1
    shared=y; //thread2 updates the value of shared variable
    printf("\nValue of shared variable updated by Thread2 is: %d\n",shared);
    sem_post(&s);
}
```

c) Write a C program that illustrate the suspending and resuming process using signals

PROGRAM:

```
#include <stdio.h>
#include <sys/types.h>
#include <signal.h>
#include <sys/wait.h>
#include <unistd.h>
int main()
{
    int pid=fork();
    if(pid==-1)
    {
        printf("\nChild not created\n");
    }
    if(pid==0)
    {
        printf("\nChild created");
    }
    else
    {
        printf("\nParent process\n");
        kill(pid,SIGSTOP);
        sleep(5);
        kill(pid,SIGCONT);
        sleep(10);
        printf("\nParent got control back\n");
        kill(pid,SIGKILL);
        wait(NULL);
    }
    return 0;
}
```

d)Write a C program that implements producer-Consumer system with two process using semaphores.

PROGRAM:

```
#include <pthread.h>
```

```
#include <semaphore.h>
```

```
#include <stdlib.h>
```

```
#include <stdio.h>
```

```
/*
```

This program provides a possible solution for producer-consumer problem using mutex and semaphore.

I have used 5 producers and 5 consumers to demonstrate the solution. You can always play with these values.

```
*/
```

```
#define MaxItems 5 // Maximum items a producer can produce or a consumer can consume
```

```
#define BufferSize 5 // Size of the buffer
```

```
sem_t empty;
```

```
sem_t full;
```

```
int in = 0;
```

```
int out = 0;
```

```
int buffer[BufferSize];
```

```
pthread_mutex_t mutex;
```

```
void *producer(void *pno)
```

```
{
```

```
    int item;
```

```
    for(int i = 0; i<MaxItems; i++) {
```

```
        item = rand(); // Produce an random item
```

```
        sem_wait(&empty);
```

```
        pthread_mutex_lock(&mutex);
```

```
        buffer[in] = item;
```

```
        printf("Producer %d: Insert Item %d at %d\n", *((int *)pno),buffer[in],in);
```

```
        in = (in+1)%BufferSize;
```

```
        pthread_mutex_unlock(&mutex);
```

```
        sem_post(&full);
```

```
    }
```

```
}
```

```
void *consumer(void *cno)
```

```

{
for(int i = 0; i<MaxItems; i++) {
sem_wait(&full);
pthread_mutex_lock(&mutex);
    int item = buffer[out];
printf("Consumer %d: Remove Item %d from %d\n",*((int *)cno),item, out);
    out = (out+1)%BufferSize;
pthread_mutex_unlock(&mutex);
sem_post(&empty);
}
}
int main()
{
pthread_t pro[5],con[5];
pthread_mutex_init(&mutex, NULL);
sem_init(&empty,0,BufferSize);
sem_init(&full,0,0);
    int a[5] = {1,2,3,4,5}; //Just used for numbering the producer and consumer
for(int i = 0; i< 5; i++) {
pthread_create(&pro[i], NULL, (void *)producer, (void *)&a[i]);
}
for(int i = 0; i< 5; i++) {
pthread_create(&con[i], NULL, (void *)consumer, (void *)&a[i]);
}
for(int i = 0; i< 5; i++) {
pthread_join(pro[i], NULL);
}
for(int i = 0; i< 5; i++) {
pthread_join(con[i], NULL);
}
pthread_mutex_destroy(&mutex);
sem_destroy(&empty);
sem_destroy(&full);
    return 0;
}

```

Experiment 5

Consider the following snapshot of a system. P0, P1, P2, P3, P4 are the processes and A, B, C, D are the resource types. The values in the table indicates the number of instances of a specific resource (for example: 3 3 2 1 under the last column indicates that there are 3 A-type, 3 B-type, 2 C-type and 1 D-type resources are available after allocating the resources to all five processes). The numbers under allocation-column indicate that those number of resources are allocated to various processes mentioned in the first column. The numbers under Max-column indicate the maximum number of resources required by the processes. For example: in 1st row under allocation-column 2 0 0 1 indicate there are 2 A-type, 0 B-type, 0 C-type and 1 D-type resources are allocated to process P0. Whereas 4 2 1 2 under Max-column indicate that process P0's maximum requirement is 4 A-type, 2 B-type, 1 C-type and 2 D-type resources.

Process	Allocation	Max	Available
	A B C D	A B C D	A B C D
P0	2 0 0 1	4 2 1 2	3 3 2 1
P1	3 1 2 1	5 2 5 2	
P2	2 1 0 3	2 3 1 6	
P3	1 3 1 2	1 4 2 4	
P4	1 4 3 2	3 6 6 5	

Answer the following questions using banker's algorithm by providing all intermediate steps

- How many instances of resources are present in the system under each type of aresource?
- Compute the Need matrix for the given snapshot of asystem.
- Verify whether the snapshot of the present system is in a safe state by demonstrating an order in which the processes may complete. iv. If a request from process P1 arrives for (1,1,0,0), can the request be granted immediately? v. If a request from process P4 arrives for (0,0,2,0), can the request be granted immediately?

PROGRAM

```
#include <stdio.h>

int main(){
    // P0, P1, P2, P3, P4 are the Process names here
    int n, m, i, j, k;
    n = 5; // Number of processes
    m = 3; // Number of resources
    int alloc[5][3] = { { 0, 1, 0 }, // P0 // Allocation Matrix
    { 2, 0, 0 }, // P1
    { 3, 0, 2 }, // P2
    { 2, 1, 1 }, // P3
    { 0, 0, 2 } }; // P4
    int max[5][3] = { { 7, 5, 3 }, // P0 // MAX Matrix
    { 3, 2, 2 }, // P1
    { 9, 0, 2 }, // P2
    { 2, 2, 2 }, // P3
    { 4, 3, 3 } }; // P4
```

```

int avail[3] = { 3, 3, 2 }; // Available Resources
int f[n], ans[n], ind = 0;
for (k = 0; k < n; k++) {
    f[k] = 0;
}
int need[n][m];
for (i = 0; i < n; i++) {
    for (j = 0; j < m; j++)
        need[i][j] = max[i][j] - alloc[i][j];
}
int y = 0;
for (k = 0; k < 5; k++) {
    for (i = 0; i < n; i++) {
        if (f[i] == 0) {

            int flag = 0;
            for (j = 0; j < m; j++) {
                if (need[i][j] > avail[j]){
                    flag = 1;
                    break;
                }
            }
            if (flag == 0) {
ans[ind++] = i;
                for (y = 0; y < m; y++)
                    avail[y] += alloc[i][y];
                f[i] = 1;
            }
        }
    }
}

printf("Following is the SAFE Sequence\n");
for (i = 0; i < n - 1; i++)
printf(" P%d ->", ans[i]);
printf(" P%d", ans[n - 1]);
return (0);

```


Experiment 6

Write a C program to simulate the following memory management technique: Paging

Program:

/*Write a C program to simulate Paging in Memory Management

```
#include <stdio.h>

int main() {
    int pages, pageSize;
    printf("Enter number of pages: ");
    scanf("%d", &pages);
    printf("Enter page size: ");
    scanf("%d", &pageSize);
    int pageTable[pages];
    printf("\nEnter frame number for each page (-1 if not allocated):\n");
    for (int i = 0; i < pages; i++) {
        printf("Page %d -> Frame: ", i);
        scanf("%d", &pageTable[i]);
    }
    int logicalAddress;
    printf("\nEnter logical address: ");
    scanf("%d", &logicalAddress);
    int pageNumber = logicalAddress / pageSize;
    int offset = logicalAddress % pageSize;
    // Check if page number is valid
    if (pageNumber >= pages) {
        printf("Invalid logical address!\n");
        return 0;
    }
    // Check if page is allocated
    if (pageTable[pageNumber] == -1) {
        printf("Page Fault! Page %d is not allocated.\n", pageNumber);
        return 0;
    }
}
```

```
    }  
  
    int frameNumber = pageTable[pageNumber];  
    int physicalAddress = frameNumber * pageSize + offset;  
    printf("\nPage Number : %d\n", pageNumber);  
    printf("Offset      : %d\n", offset);  
    printf("Frame Number: %d\n", frameNumber);  
    printf("Physical Address = %d\n", physicalAddress);  
    return 0;  
}
```

Experiment 7

a) Write a C program that takes one or more file/directory names as command line input and reports following information i) File Type ii) Number of Links iii) Time of last Access iv) Read, write and execute permissions.

PROGRAM:

```
#include<stdio.h>

#include<sys/stat.h>

#include<time.h>

int main(int argc, char *argv[])

{

    int i,j;

    struct stat a;

    for(i=1;i<argc;i++)

    {

        printf("%s\n",argv[i]);

        stat(argv[i],&a);

        if(S_ISDIR(a.st_mode))

        {

            printf("Is a directory\n");

        }

        else

        {

            printf("Is a regular file\n");

        }

        printf("FILE PROPERTIES\n");

        printf("Inodenumnber: %d\n",a.st_ino);

        printf("No of Links: %d\n",a.st_nlink);

        printf("Last access time: %s\n",asctime(localtime(&a.st_atime)));

    }

}
```

```
    printf("Permission Flag: %o\n",a.st_mode%512);  
    printf("Size : %d\n",a.st_size);  
    }  
    return 0;  
}
```

b)Write a C program to list every file in directory, its inode number and file name

PROGRAM:

```
#include <fcntl.h>
#include <stdio.h>
#include <dirent.h>
#include <sys/stat.h>
int main(int argc,char *argv[])
{
    DIR *dirop;
    struct dirent *dired;
    if(argc!=2)
    {
        printf("Invalid number of arguments\n");
    }
    else if((dirop=opendir(argv[1]))==NULL)
        printf("Cannot open Directory\n");
    else
    {
        printf("Inode\tFile Name\n");
        while((dired=readdir(dirop))!=NULL)
            printf("%ld\t%s\n ",dired->d_ino,dired->d_name);
        closedir(dirop);
    }
    return 0;
}
```